

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

Про виконання лабораторної роботи №4
“Основні типи змінних. Операції та їх пріоритет”

Виконала:
Студентка ФЕП-13с
Пономаренко Д.С.

Перевірів
проф. Крупич О. М.

Львів 2026

Мета роботи:

Вивчити поняття: компіляції, функцій вводу/виводу.

Хід роботи

- ❖ Опрацювати і засвоїти матеріал наведений в теоретичних відомостях.
- ❖ Написати програму, котра ілюструє використання всіх арифметичних і логічних операторів мови Сі, для логічних побітових операцій використовувати представлення чисел в двійковому форматі.

```
main.c x
Project5 (Global Scope)
1 #include <stdio.h>
2 #include <windows.h>
3
4 void print_binary(unsigned char num) {
5     for (int i = 7; i >= 0; i--) {
6         printf("%d", (num >> i) & 1);
7     }
8     printf("\n");
9 }
10 int main() {
11     SetConsoleOutputCP(CP_UTF8);
12     SetConsoleCP(CP_UTF8);
13     int a = 10, b = 3;
14
15     printf("=== Арифметичні оператори ===\n");
16     printf("a + b = %d\n", a + b);
17     printf("a - b = %d\n", a - b);
18     printf("a * b = %d\n", a * b);
19     printf("a / b = %d\n", a / b);
20     printf("a %% b = %d\n", a % b);
21 }
```

```
main.c x
Project5 (Global Scope)
19 printf("a / b = %d\n", a / b);
20 printf("a %% b = %d\n", a % b);
21
22 printf("\n=== Логічні оператори ===\n");
23 printf("(a > 5) && (b < 5): %d\n", (a > 5) && (b < 5));
24 printf("(a < 5) || (b < 5): %d\n", (a < 5) || (b < 5));
25 printf("! (a == b): %d\n", !(a == b));
26
27 unsigned char x = 12;
28 unsigned char y = 5;
29
30 printf("\n=== Побітові операції у двійковому форматі ===\n");
31 printf("x = "); print_binary(x);
32 printf("y = "); print_binary(y);
33 printf("x & y = "); print_binary(x & y);
34 printf("x | y = "); print_binary(x | y);
35 printf("x ^ y = "); print_binary(x ^ y);
36 printf("~x = "); print_binary((unsigned char)~x);
37 printf("x << 1 = "); print_binary(x << 1);
38 printf("x >> 1 = "); print_binary(x >> 1);
39 }
```

```
Microsoft Visual Studio Debu...
=== Арифметичні оператори ===
a + b = 13
a - b = 7
a * b = 30
a / b = 3
a % b = 1

=== Логічні оператори ===
(a > 5) && (b < 5): 1
(a < 5) || (b < 5): 1
!(a == b): 1

=== Побітові операції у двійковому форматі ===
x      = 00001100
y      = 00000101
x & y   = 00000100
x | y   = 00001101
x ^ y   = 00001001
~x      = 11110011
x << 1  = 00011000
x >> 1  = 00000110

C:\Users\user\source\repos\Project5\x64\Debug\Project5.exe (process 20464) exited with code 0 (0x0).
Press any key to close this window . . .
```

У цій програмі я реалізувала роботу з трьома типами операторів у мові С: арифметичними, логічними та побітовими. На початку коду я створила функцію `print_binary`, яка приймає число та виводить його у двійковому 8-бітному вигляді, по черзі перевіряючи кожен біт за допомогою зсуву. Також у `main` я додала налаштування `CP_UTF8`, щоб у консолі коректно відображався

український текст. У першому блоці я виконала базові арифметичні операції для двох чисел ($a = 10$ і $b = 3$). У другому блоці я перевірила логічні оператори. Тут 1 означає істину, а 0 — хибу. Операція «І» (&&) повернула 1, тому що обидві умови ($10 > 5$) і ($3 < 5$) виявилися правильними. Операція «АБО» (||) також дала 1, бо умові ($3 < 5$) достатньо хоча б одного правильного твердження. Операція «НЕ» (!) повернула 1, адже рівність ($10 == 3$) є хибною, а її заперечення дає істину. У третьому блоці я виконала побітові операції над числами $x = 12$ (00001100) та $y = 5$ (00000101). Побітове «І» (&) дало 00000100 (4), залишивши одиниці тільки там, де вони були в обох числах. Побітове «АБО» (|) дало 00001101 (13), об'єднавши одиниці з двох чисел. Виключне «АБО» (^) повернуло 00001001 (9), залишивши одиниці лише там, де біти відрізнялися. Операція ~x інвертувала всі біти, змінивши нулі на одиниці (11110011). Зсув ліворуч ($x << 1$) перемістив біти на один крок уліво (00011000), що помножило число на 2 (вийшло 24), а зсув праворуч ($x >> 1$) перемістив біти вправо (00000110), що поділило число на 2 (вийшло 6).

- ❖ Написати програму з введенням числа у змінну і виведенням її адреси та значення через вказівник.

```
Microsoft Visual Studio Debug Console
Введіть ціле число: 35

--- Результати ---
Значення змінної (прямий доступ): 35
Адреса змінної (&number): 0000005DF7DDF5D4
Адреса, збережена у вказівнику (ptr): 0000005DF7DDF5D4
Значення через розіменування вказівника (*ptr): 35

C:\Users\user\source\repos\Project5\x64\Debug\Project5.exe (process 27020) exited with code 0 (0x0).
Press any key to close this window . . .
```

```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
Debug x64 Local Windows Debugger

main.c
Project5 (Global Scope)
1 #define _CRT_SECURE_NO_WARNINGS
2
3 #include <stdio.h>
4 #include <windows.h>
5
6 int main() {
7     SetConsoleOutputCP(CP_UTF8);
8     SetConsoleCP(CP_UTF8);
9     int number;
10    int* ptr = &number;
11
12    printf("Введіть ціле число: ");
13    scanf("%d", &number);
14
15    printf("\n--- Результати ---\n");
16    printf("Значення змінної (прямий доступ): %d\n", number);
17    printf("Адреса змінної (&number): %p\n", (void*)&number);
18    printf("Адреса, збережена у вказівнику (ptr): %p\n", (void*)ptr);
19    printf("Значення через розіменування вказівника (*ptr): %d\n", *ptr);
20
21    return 0;
22 }
```

У цій програмі я реалізувала роботу з вказівниками. Я оголосила цілочисельну змінну `number` та створила вказівник `ptr`, у який зберегла її адресу за допомогою оператора `&number`. Після того, як користувач ввів число з клавіатури через `scanf`, я вивела чотири параметри:

1. Значення змінної напряму через `number`.
2. Адресу цієї змінної у пам'яті комп'ютера за допомогою `&number`.
3. Адресу, яку зберігає вказівник `ptr` (вона повністю збіглася з адресою змінної).
4. Значення змінної через розіменування вказівника (`*ptr`), що дало той самий результат, що й прямий доступ.

Таким чином я перевірила, що вказівник дійсно посилається на комірку пам'яті цієї змінної і дозволяє отримувати її значення.

❖ Написати програму розв'язку квадратного рівняння, коефіцієнти якого вводяться з клавіатури.

```
main.c x
Project5 (Global Scope)
1 #define _CRT_SECURE_NO_WARNINGS
2
3 #include <stdio.h>
4 #include <windows.h>
5 #include <math.h>
6
7 int main() {
8     SetConsoleOutputCP(CP_UTF8);
9     SetConsoleCP(CP_UTF8);
10    double a, b, c; double discriminant, x1, x2, realPart, imagPart;
11    printf("Введіть коефіцієнти a, b і c (через пробіл): ");
12    scanf("%lf %lf %lf", &a, &b, &c);
13
14    if (a == 0) {
15        printf("Це не квадратне рівняння (a не може дорівнювати 0).\n");
16        return 0;
17    }
18
19    discriminant = b * b - 4 * a * c;
20
21    if (discriminant > 0) {
```

```
main.c x
Project5 (Global Scope)
19    discriminant = b * b - 4 * a * c;
20
21    if (discriminant > 0) {
22        x1 = (-b + sqrt(discriminant)) / (2 * a);
23        x2 = (-b - sqrt(discriminant)) / (2 * a);
24        printf("Дійсні та різні корені:\n");
25        printf("x1 = %.2lf\n", x1);
26        printf("x2 = %.2lf\n", x2);
27    }
28    else if (discriminant == 0) {
29        x1 = -b / (2 * a);
30        printf("Дійсні та однакові корені:\n");
31        printf("x1 = x2 = %.2lf\n", x1);
32    }
33    else {
34        printf("\nРівняння не має дійсних коренів.\n");
35    }
36
37    return 0;
38 }
```

```
Microsoft Visual Studio Debug Console
Введіть коефіцієнти a, b і c (через пробіл): 2 -4 -6
Дійсні та різні корені:
x1 = 3.00
x2 = -1.00

C:\Users\user\source\repos\Project5\x64\Debug\Project5.exe (process 27396) exited with code 0 (0x0).
Press any key to close this window . . .
```

У цій програмі я написала алгоритм для знаходження коренів квадратного рівняння вигляду $ax^2 + bx + c = 0$. Спочатку я зробила зчитування трьох коефіцієнтів `a`, `b` і `c`, які вводить користувач. Далі я додала перевірку: якщо `a == 0`, програма виводить повідомлення про помилку, оскільки рівняння перестає бути квадратним. Після цього я обчислила дискримінант за формулою

$D = b^2 - 4ac$ і додала розгалуження через if / else if / else:

1. **Якщо $D > 0$:** програма обчислює та виводить два різних корені x_1 та x_2 за стандартною формулою з використанням функції `sqrt()`.
2. **Якщо $D = 0$:** програма знаходить один корінь (або два однакові) за формулою $x = -b / (2a)$.
3. **Якщо $D < 0$:** програма виводить повідомлення про те, що рівняння не має дійсних коренів.

Висновок

Під час виконання лабораторної роботи я на практиці засвоїла роботу з основними типами операторів у мові C. Я навчилася використовувати арифметичні, логічні та побітові оператори, а також реалізувала виведення чисел у двійковому форматі. Окрім цього, я закріпила навички роботи з вказівниками для доступу до адреси та значення змінних у пам'яті. Також я написала програму для розв'язання квадратного рівняння з використанням умовних операторів та математичних функцій. Усі поставлені завдання виконано повністю, а результати роботи програм є коректними.